



PLAYWRIGHT VS SELENIUM

A Practical Comparison

Architecture, speed, debugging, and where each tool still wins



HOW THEY TALK TO THE BROWSER

Architecture

Selenium

- Uses the WebDriver protocol — a series of discrete HTTP requests to a driver process (ChromeDriver, GeckoDriver, etc.)
- Each command is a round trip: request, response, repeat
- An extra abstraction layer sits between your test code and the real browser

Playwright

- Opens one persistent WebSocket connection directly into the browser via its native protocol (CDP for Chromium, equivalents for Firefox/WebKit)
- Commands stream over that one connection instead of separate requests
- Fewer moving parts between your code and the browser



THE #1 SOURCE OF FLAKY TESTS

Waiting Strategy

Selenium

- No built-in actionability checks — an element can exist but still not be clickable
- Flakiness is usually solved with explicit WebDriverWait + custom expected_conditions
- Easy to fall back on sleep() under deadline pressure, which is slow and still not reliable

Playwright

- Auto-waits before every action: attached, visible, stable, enabled, and able to receive events
- Assertions (expect) poll the actual condition until it's true or times out
- Same idea applies to a recent open-source meta-analysis: Playwright suites ~0.8% flake rate vs ~4.1% for Selenium, with wait strategy the main driver



RUNNING THE SUITE

Speed & Parallelization

Selenium

- Parallelization works (Selenium Grid, or pytest-xdist style runners) but takes more infrastructure to set up well
- Each browser session is heavier to spin up via the driver layer

Playwright

- Multiple isolated browser contexts (think: separate incognito sessions) inside one browser instance — cheap to create
- Built-in test runner parallelizes across workers with no extra grid setup



WHEN SOMETHING FAILS

Debugging & Tooling

Selenium

- Debugging usually means a stack trace plus whatever screenshot/logging you wired up yourself
- Selenium IDE exists for basic recording, but it's a separate, fairly limited tool
- No built-in equivalent to a full execution trace

Playwright

- Trace Viewer: full timeline of DOM snapshots, network calls, console logs, and screenshots per action, after the fact
- Codegen records your clicks and writes the locator code live
- VS Code extension adds inline run/debug, a test sidebar, and AI-assisted fix suggestions for failures



WHERE SELENIUM STILL LEADS

Browser & Language Breadth

Selenium

- Broader official language support: Java, Python, C#, Ruby, JavaScript, Kotlin, and more
- Real browser coverage via WebDriver, including older/enterprise browsers some industries still require
- Massive, decades-deep ecosystem of grids, cloud providers, and integrations

Playwright

- Officially supports 4 languages: JavaScript/TypeScript, Python, Java, .NET
- Tests against Chromium, Firefox, and WebKit — WebKit approximates Safari but isn't literal Safari on macOS
- Younger ecosystem; growing fast, but fewer third-party integrations than Selenium has accumulated



BOTTOM LINE

Neither tool is wrong — they fit different situations

Reach for Selenium when

- You already maintain a large, working Selenium suite — rewriting has a real cost
- You need language support outside JS/TS, Python, Java, .NET
- You need coverage of legacy or niche browsers Playwright doesn't target

Reach for Playwright when

- You're starting something new and want auto-waiting and less flake-fighting out of the box
- Fast feedback loops and built-in parallelization matter
- Debugging tooling (trace viewer, codegen, VS Code integration) is a priority for the team